1. Introducción

1.1 Contexto del proyecto

Konys Pizza es una pizzería ubicada en la Cra. 36 con calle 41 de Bucaramanga (Santander), conocida como 'la pizza más icónica de la ciudad'. Cuenta con más de 34.900 seguidores en Instagram, atiende en dos jornadas (de 12:30 a 2:30 p. m. y de 5:00 a 11:00 p. m.), ofrece servicio de domicilios y llega a eventos. Actualmente las reservas de mesa se gestionan de manera manual, ya sea por llamada, por mensajes de WhatsApp o Instagram, o de forma presencial. Esto genera problemas como mesas reservadas dos veces, tiempos de espera prolongados y pérdida de clientes por falta de visibilidad de la disponibilidad en tiempo real. El desarrollo de una aplicación web de reservas busca digitalizar y optimizar este proceso.

> 💡 **Nota pedagógica**
> Un buen documento de proyecto parte siempre del contexto real del negocio. Antes de escribir una sola línea de código, es fundamental entender el problema que se va a resolver.

2. Descripción del Problema

2.1 Situación actual

Actualmente, Konys Pizza maneja las reservas de sus clientes de manera completamente manual. Los clientes llaman por teléfono, escriben por WhatsApp o se presentan directamente en el local para separar una mesa. Este proceso presenta múltiples deficiencias que afectan tanto la operación del negocio como la experiencia del cliente.

2.2 Problemas identificados

- **Doble reserva de mesas:** cuando dos clientes reservan la misma mesa en el mismo horario sin que nadie lo note.
- **Falta de visibilidad:** el cliente no puede saber desde su casa qué mesas y horarios están disponibles.
- **Carga operativa:** el administrador debe gestionar manualmente cada reserva, lo que consume tiempo valioso en horas de alta demanda.
- **Pérdida de clientes:** quienes intentan comunicarse fuera del horario de atención, o cuando la línea está ocupada, no pueden reservar.
- **Sin historial:** no existe registro digital de las reservas pasadas ni de la demanda por día y horario.

> 📌 **Ejemplo real del problema**
> Imagina que Juan escribe a las 3 p. m. pidiendo una mesa para cuatro personas el sábado a las 7:00 p. m. El administrador lo anota en una libreta. Luego María llama y también pide esa mesa para el sábado a las 7:00 p. m., y el administrador, ocupado con los pedidos a domicilio, la agenda también. El resultado: dos grupos en la puerta, un cliente molesto y una mala reseña en Google.

2.3 Impacto del problema

El impacto de esta situación se puede medir en tres dimensiones principales:

- **Impacto económico:** pérdida de ingresos por mesas vacías, reservas dobles o clientes que no logran reservar.
- **Impacto en la reputación:** una mala experiencia reduce las recomendaciones y la calificación del restaurante en Google.
- **Impacto operativo:** el administrador invierte tiempo en coordinación manual en lugar de en la gestión del negocio.

3. Objetivos del Sistema

3.1 Objetivo general

Desarrollar una aplicación web que permita a los clientes de Konys Pizza reservar mesas de forma autónoma, visualizando la disponibilidad en tiempo real, y que brinde al administrador herramientas para gestionar las mesas, los horarios y las reservas de manera eficiente.

3.2 Objetivos específicos

1. Implementar una interfaz visual intuitiva donde el cliente pueda ver las mesas del restaurante y seleccionar una para su reserva.
2. Permitir que el cliente elija fecha, hora y número de personas, y confirme su reserva sin necesidad de llamar o desplazarse.
3. Mostrar la disponibilidad de cada mesa (libre u ocupada) por franjas horarias.
4. Proveer al administrador un panel para registrar mesas, definir los horarios de atención y consultar las reservas.
5. Persistir todos los datos en una base de datos en Supabase para garantizar la disponibilidad de la información.
6. Desplegar la aplicación en Vercel con el código fuente gestionado en un repositorio de GitHub (mínimo 5 commits).
7. Documentar el diseño mediante mockups de baja fidelidad y presentar el trabajo en un video explicativo.

> 💡 **Consejo para el estudiante**
> Los objetivos específicos son la columna vertebral del proyecto. Cada objetivo debe traducirse en al menos un requisito funcional. Si un objetivo no se puede medir o validar, no está bien redactado. Usa verbos de acción: 'implementar', 'permitir', 'mostrar', 'proveer'.

4. Alcance del Proyecto

4.1 Lo que SÍ incluye el proyecto

- Módulo de cliente: visualización de mesas, selección de mesa, elección de fecha/hora y confirmación de la reserva.
- Módulo de administrador: registro y gestión de mesas, definición de horarios y visualización de reservas.
- Consulta de la carta digital (pizzas, entradas, postres y bebidas) en modo lectura.
- Interfaz de disponibilidad: visualización clara de franjas libres y ocupadas.
- Almacenamiento de datos con Supabase.
- Mockups de baja fidelidad de las pantallas principales.
- Despliegue funcional en Vercel con código en GitHub (mínimo 5 commits).

4.2 Lo que NO incluye el proyecto (fuera del alcance)

- Sistema de pagos en línea o anticipos de reserva.
- Pedidos a domicilio en línea (se mantiene el canal actual por WhatsApp).
- Aplicación móvil nativa (iOS o Android).
- Autenticación de clientes (registro de usuario).
- Notificaciones por correo electrónico, SMS o WhatsApp.
- Reportes o estadísticas de ventas.
- Gestión de inventario o de insumos de cocina.

> ⚠️ **Por qué es importante definir el alcance**
> Definir claramente qué está dentro y qué está fuera del alcance evita el fenómeno conocido como 'scope creep' (expansión descontrolada del alcance). En proyectos reales, no delimitar el alcance es una de las principales causas de retrasos y sobrecostos. Las funcionalidades fuera del alcance pueden considerarse mejoras futuras.

5. Identificación de Actores

Un actor es cualquier persona, sistema o entidad que interactúa con la aplicación. Identificarlos correctamente permite definir qué puede hacer cada uno y qué permisos necesita.

Actor	Descripción y rol
Cliente	Persona que desea reservar una mesa en Konys Pizza. Accede al módulo de reservas sin necesidad de autenticarse. Puede ver la disponibilidad, consultar la carta y confirmar su reserva.
Administrador	Persona encargada de la gestión operativa del restaurante. Accede a un panel de administración para registrar mesas, definir horarios, ver reservas y administrar la carta. Se autentica con credenciales.
Sistema (Supabase)	Actor externo que actúa como base de datos. Recibe y responde consultas de la aplicación para leer y escribir datos de mesas, horarios, reservas y platos.

> 💡 **Nota**
> En este proyecto no se implementa un módulo de autenticación complejo para el cliente. El administrador puede proteger su panel con una clave simple en el front-end como medida básica de seguridad, aunque en producción se recomendaría una autenticación real.

6. Requisitos Funcionales

Los requisitos funcionales describen qué debe hacer el sistema. Se organizan por módulo y se numeran para facilitar su trazabilidad.

📌 **¿Qué es la trazabilidad?**

Trazabilidad significa que cada requisito puede rastrearse desde su origen (necesidad del negocio) hasta su implementación (código) y prueba (caso de prueba). Es una buena práctica en ingeniería de software identificar cada requisito con un código único como RF-01.

6.1 Módulo Cliente

RF-01: Visualización de mesas disponibles

El sistema deberá mostrar en la página principal una representación visual de las mesas del restaurante. Cada mesa mostrará su nombre, la zona (terraza o salón), su capacidad y su estado general. El cliente podrá ver de un vistazo cuántas mesas existen y cuáles tienen cupo.

RF-02: Selección de mesa

El cliente podrá hacer clic en una mesa disponible para seleccionarla. Al seleccionarla, el sistema mostrará los detalles de esa mesa y los horarios disponibles para la fecha seleccionada.

RF-03: Elección de fecha

El sistema deberá ofrecer un calendario o selector de fecha que permita al cliente elegir el día de su reserva. El sistema solo mostrará días habilitados dentro del horario de operación del restaurante.

RF-04: Visualización de horarios disponibles

Una vez seleccionadas la mesa y la fecha, el sistema deberá mostrar los bloques de tiempo (slots) disponibles y no disponibles para ese día y mesa. Los slots ocupados estarán visualmente diferenciados de los disponibles (ej: gris para ocupado, blanco o verde para libre).

RF-05: Reserva de mesa

El cliente podrá seleccionar un slot disponible y confirmar su reserva. El sistema deberá solicitar el nombre del cliente, el número de personas (que no podrá superar la capacidad de la mesa) y opcionalmente un número de contacto. Al confirmar, el slot pasará a estado 'ocupado' y la reserva quedará registrada en la base de datos.

RF-06: Confirmación de reserva

Tras completar la reserva, el sistema mostrará en pantalla un mensaje de confirmación con los datos: mesa, fecha, hora, número de personas y nombre del cliente. Este mensaje servirá como comprobante visual de la reserva.

RF-07: Consulta de la carta digital

El cliente podrá consultar la carta del restaurante organizada por categorías (pizzas, entradas, postres y bebidas), con nombre, descripción y precio de cada plato. Los platos marcados como no disponibles no se mostrarán.

6.2 Módulo Administrador

RF-08: Acceso al panel de administración

El sistema deberá contar con una sección de administración accesible mediante una URL específica (ej: /admin). El acceso deberá estar protegido por una contraseña simple configurada en el sistema.

RF-09: Registro de mesas

El administrador podrá registrar nuevas mesas ingresando: nombre o número, zona (terraza o salón), capacidad de personas y, opcionalmente, una foto. Cada mesa quedará disponible para recibir reservas.

RF-10: Listado de mesas

El sistema mostrará una lista de todas las mesas registradas con su información básica: nombre, zona, capacidad y estado actual (activa / inactiva). El administrador podrá ver este listado desde el panel de administración.

RF-11: Gestión de horarios

El administrador podrá definir o modificar el horario de atención por día de la semana y la duración de cada reserva. Por ejemplo, indicar que la pizzería atiende en dos jornadas: de 12:30 p. m. a 2:30 p. m. y de 5:00 p. m. a 11:00 p. m., con reservas de 60 minutos. El sistema usará esta información para generar los slots disponibles.

RF-12: Visualización de reservas

El administrador podrá consultar todas las reservas registradas, con la posibilidad de filtrar por fecha o por mesa. Cada reserva mostrará: nombre del cliente, mesa, fecha, hora, número de personas y estado.

RF-13: Cambio de estado de mesa

El administrador podrá marcar una mesa como 'activa' o 'inactiva'. Esta funcionalidad sirve para casos como mantenimiento, eventos privados o cierre temporal de la terraza. Cuando una mesa está inactiva, sus slots no aparecerán en el módulo de cliente.

RF-14: Gestión de la carta

El administrador podrá agregar, editar, ocultar y eliminar platos de la carta digital, indicando nombre, categoría, descripción, precio y disponibilidad.

7. Requisitos No Funcionales

Los requisitos no funcionales describen cómo debe comportarse el sistema, no qué debe hacer. Incluyen aspectos de rendimiento, seguridad, usabilidad y mantenibilidad.

Requisito	Descripción
RNF-01 · Usabilidad	La interfaz debe ser intuitiva y no requerir capacitación. Un cliente sin experiencia técnica debe poder reservar una mesa en menos de 3 pasos sin necesidad de instrucciones adicionales.
RNF-02 · Responsividad	La aplicación debe adaptarse correctamente a dispositivos móviles, tablets y computadoras de escritorio. Se considera que la mayoría de los clientes accederán desde su teléfono celular.
RNF-03 · Rendimiento	Las páginas deben cargarse en menos de 3 segundos bajo condiciones normales de red. Las consultas a Supabase deben responder en menos de 1 segundo para una buena experiencia de usuario.
RNF-04 · Disponibilidad	La aplicación debe estar disponible el 99% del tiempo. Al usar Vercel y Supabase como infraestructura, ambos ofrecen alta disponibilidad dentro de sus planes gratuitos.
RNF-05 · Seguridad	Los datos de los clientes (nombre y contacto) deben transmitirse de forma segura mediante HTTPS. El acceso al panel de administración debe estar protegido para evitar modificaciones no autorizadas.
RNF-06 · Mantenibilidad	El código debe estar organizado en archivos separados por funcionalidad (HTML, CSS, JS) con comentarios claros. Esto facilita que otro desarrollador pueda entender y modificar el sistema en el futuro.
RNF-07 · Compatibilidad	La aplicación debe funcionar correctamente en los navegadores modernos más populares: Google Chrome, Mozilla Firefox, Microsoft Edge y Safari, en sus versiones actuales.
RNF-08 · Escalabilidad	La arquitectura debe permitir agregar nuevas mesas, zonas o platos sin necesidad de reestructurar la base de datos o refactorizar el código de manera significativa.
RNF-09 · Trazabilidad del código	El repositorio de GitHub debe reflejar la evolución del proyecto con un mínimo de 5 commits descriptivos, distribuidos a lo largo del desarrollo.

8. Casos de Uso

Un caso de uso describe la interacción entre un actor y el sistema para lograr un objetivo específico. A continuación, se presentan los casos de uso principales del sistema.

8.1 Resumen de casos de uso

Código	Caso de uso	Actor	Descripción breve
CU-01	Visualizar disponibilidad	Cliente	El cliente accede a la app y ve las mesas disponibles.
CU-02	Reservar mesa	Cliente	El cliente selecciona mesa, fecha, hora y confirma la reserva.
CU-03	Consultar carta	Cliente	El cliente revisa los platos y precios del restaurante.
CU-04	Acceder al panel admin	Administrador	El admin ingresa al panel con contraseña.
CU-05	Registrar mesa	Administrador	El admin crea un nuevo registro de mesa en el sistema.
CU-06	Gestionar horario	Administrador	El admin define o modifica el horario de atención.
CU-07	Ver reservas	Administrador	El admin consulta el listado de reservas.
CU-08	Cambiar estado de mesa	Administrador	El admin marca una mesa como activa o inactiva.

8.2 Caso de uso detallado: CU-02 – Reservar Mesa

A continuación, se desarrolla el caso de uso más importante del sistema de manera detallada.

Campo	Detalle
Código	CU-02
Nombre	Reservar Mesa
Actor principal	Cliente
Precondición	La aplicación está disponible. Existe al menos una mesa activa con horarios definidos.

Campo	Detalle
Flujo principal	1. El cliente abre la aplicación web. 2. El sistema muestra las mesas del restaurante. 3. El cliente selecciona una mesa. 4. El sistema muestra un calendario de fechas disponibles. 5. El cliente selecciona una fecha. 6. El sistema muestra los slots de hora disponibles y ocupados. 7. El cliente selecciona un slot disponible. 8. El sistema muestra un formulario para ingresar nombre, número de personas y (opcional) contacto. 9. El cliente ingresa sus datos y confirma la reserva. 10. El sistema guarda la reserva en Supabase y marca el slot como ocupado. 11. El sistema muestra un mensaje de confirmación con los datos de la reserva.
Flujo alternativo	En el paso 7: si el cliente selecciona un slot ya ocupado, el sistema muestra un mensaje de error y le pide seleccionar otro horario. En el paso 9: si el número de personas supera la capacidad de la mesa, el sistema informa el máximo permitido y solicita corregirlo.
Postcondición	La reserva queda registrada en la base de datos y el slot aparece como ocupado en la interfaz.

9. Propuesta de Arquitectura

La arquitectura de una aplicación define cómo se organizan y comunican sus componentes. Para este proyecto se propone una arquitectura simple de tres capas, apropiada para el stack tecnológico elegido.

9.1 Arquitectura en tres capas

Capa	Descripción
Capa de Presentación (Front-end)	Contiene toda la interfaz de usuario. Está construida con HTML, CSS y JavaScript vanilla. Es lo que el usuario ve y con lo que interactúa directamente en el navegador. No hay lógica de negocio compleja aquí, solo presentación y control de eventos.
Capa de Lógica (Front-end JS)	En este proyecto, al no existir un back-end propio, la lógica de negocio reside en los archivos JavaScript del cliente. Aquí se gestionan los estados de la UI, se validan los formularios y se construyen las consultas a la API de Supabase.
Capa de Datos (Supabase)	Supabase actúa como back-end de datos. Proporciona una API REST que el JavaScript del front-end consume directamente para leer y escribir datos. La base de datos almacena las colecciones de mesas, horarios, reservas y platos.

9.2 Flujo de comunicación

El flujo de datos entre las capas es el siguiente:

1. El navegador del usuario carga los archivos HTML, CSS y JS desde Vercel.
2. El JavaScript en el navegador realiza llamadas HTTP (fetch) a la API de Supabase.
3. Supabase procesa la solicitud, consulta su base de datos y devuelve los datos en formato JSON.
4. El JavaScript procesa la respuesta y actualiza la interfaz de usuario de forma dinámica.

> 💡 **¿Por qué no hay un back-end propio?**
> Para este proyecto académico, se optó por no desarrollar un servidor back-end (como Node.js o Python/Flask) para simplificar el stack. Supabase provee una API REST segura que puede consumirse directamente desde el navegador con la clave de API correcta. Esta arquitectura se denomina 'JAMstack' (JavaScript, APIs, Markup) y es muy común en proyectos modernos de pequeña y mediana escala.

9.3 Stack tecnológico

Tecnología	Uso
HTML5	Estructura y marcado semántico de las páginas web.
CSS3	Estilizado, diseño visual y responsividad (Flexbox/Grid).
JavaScript (ES6+)	Lógica del cliente, manejo del DOM, llamadas a la API.
Supabase (SDK JS)	Base de datos como servicio. Se accede mediante su SDK o API REST.
GitHub	Control de versiones y repositorio del código fuente.
Vercel	Plataforma de despliegue continuo. Se conecta al repositorio de GitHub.
Figma / Balsamiq / papel	Elaboración de los mockups de baja fidelidad.

10. Modelo de Datos

El modelo de datos describe cómo se organizará y almacenará la información en Supabase. Los datos se organizan en colecciones de registros con campos definidos.

> 📌 **Base de datos relacional vs no relacional**
> En una base de datos relacional (como MySQL o PostgreSQL), los datos se organizan en tablas con filas y columnas. En una base de datos no relacional, los datos se almacenan como colecciones de objetos JSON, lo que da más flexibilidad para datos con estructura variable. Supabase está construido sobre PostgreSQL; para este proyecto se usará su interfaz de tablas/colecciones y se podrán guardar campos en formato JSON cuando haga falta.

10.1 Colección: mesas

Almacena la información de cada mesa del restaurante.

Campo	Descripción
id	Identificador único de la mesa (generado automáticamente por Supabase).
nombre	Nombre o número de la mesa. Ej: 'Mesa 3'.
zona	Ubicación de la mesa. Ej: 'terraza' o 'salon'.
capacidad	Número máximo de personas. Ej: 4.
activa	Boolean que indica si la mesa está disponible para reservas. Ej: true/false.
foto_url	URL opcional de la foto de la mesa o de su zona para mostrar en la interfaz.
creado_en	Timestamp de cuando fue registrada la mesa en el sistema.

10.2 Colección: horarios

Define el horario de atención del restaurante por día de la semana.

Campo	Descripción
id	Identificador único del registro de horario.
dia_semana	Día de la semana. Ej: 'lunes', 'martes', etc.
jornada	Franja del día: 'almuerzo' (12:30 p. m. – 2:30 p. m.) o 'noche' (5:00 p. m. – 11:00 p. m.). Un mismo día puede tener dos registros.
hora_inicio	Hora de apertura de la jornada. Ej: '12:30'.
hora_fin	Hora de cierre de la jornada. Ej: '14:30'.
duracion_slot	Duración en minutos de cada reserva. Ej: 60.
activo	Boolean que indica si este horario está vigente.

10.3 Colección: reservas

Registra cada reserva realizada por un cliente.

Campo	Descripción
id	Identificador único de la reserva (generado automáticamente).
mesa_id	Referencia a la mesa reservada.
cliente_nombre	Nombre del cliente que realizó la reserva.
cliente_contacto	Número de teléfono u otro medio de contacto (opcional).
num_personas	Cantidad de personas de la reserva. No puede superar la capacidad de la mesa.
fecha	Fecha de la reserva. Ej: '2026-11-15'.
hora_inicio	Hora de inicio de la reserva. Ej: '19:00'.
hora_fin	Hora estimada de fin. Ej: '20:00'.
estado	Estado de la reserva: 'confirmada', 'cancelada' o 'completada'.
creado_en	Timestamp de cuando se realizó la reserva.

10.4 Colección: platos

Representa los platos de la carta digital del restaurante.

Campo	Descripción
id	Identificador único del plato.
nombre	Nombre del plato. Ej: 'Pizza Konys Especial'.
categoria	Categoría: 'pizzas', 'entradas', 'postres', 'bebidas'.
descripcion	Descripción breve de los ingredientes.
precio	Precio en pesos colombianos (COP).
disponible	Boolean que indica si el plato se muestra en la carta.

11. Diseño de Interfaz

Esta sección describe la estructura y disposición de las pantallas de la aplicación y presenta los mockups de baja fidelidad que guían el diseño visual durante el desarrollo.

11.1 Página principal – Vista del cliente

La página principal es lo primero que ve el cliente al ingresar a la aplicación. Debe ser atractiva, limpia y directa al grano.

- **Encabezado (Header):** logotipo de Konys Pizza en la esquina superior izquierda, menú de navegación (Inicio, Mesas, Carta, Reservar, Contacto) y un botón discreto de acceso al panel de administración.
- **Sección hero:** una franja superior con el nombre del restaurante, un eslogan ('La pizza más icónica de la ciudad') y un botón de llamada a la acción: 'Reserva tu mesa'.
- **Sección de mesas:** representación visual tipo cards (tarjetas) de cada mesa. Cada card muestra la foto, el nombre, la zona, la capacidad y el estado de disponibilidad. Al hacer clic en una card, se activa el flujo de reserva.
- **Flujo de reserva (modal o sección deslizable):** una vez seleccionada la mesa, aparece un selector de fecha, luego una grilla de slots horarios (visual y coloreada) y finalmente un formulario de confirmación.
- **Pie de página (Footer):** dirección (Cra. 36 con calle 41, Bucaramanga), teléfono/WhatsApp y redes sociales.

11.2 Panel de administración

Accesible desde /admin, esta interfaz es de uso exclusivo del administrador. Debe ser funcional y organizada.

- **Pantalla de acceso:** formulario de contraseña para ingresar al panel.
- **Menú lateral:** navegación entre las secciones del panel: Mesas, Horarios, Reservas y Carta.
- **Sección Mesas:** tabla con la lista de mesas registradas. Botón para agregar nueva mesa (abre un formulario). Botones de acción por mesa: Editar, Borrar y toggle Activa/Inactiva.
- **Sección Horarios:** tabla editable con el horario de atención por día de la semana. Cada fila representa un día con su hora de inicio, fin y duración de la reserva.
- **Sección Reservas:** tabla de reservas con filtros por fecha y por mesa. Cada fila muestra: cliente, mesa, fecha, hora, personas y estado.
- **Sección Carta:** tabla de platos con acciones para agregar, editar u ocultar.

11.3 Paleta de colores y tipografía (sugerencia)

Elemento	Descripción
Color primario	Negro o gris oscuro (#0F2B3D, azul petróleo oscuro) – coincide con el fondo del logo de Konys y transmite sobriedad.
Color de acento	Naranja (#F28C28) – color del logo de Konys; resalta los elementos clave (botones, títulos).
Color de fondo	Blanco o gris muy claro – para el contenido y el panel de administración.
Tipografía	Sans-serif moderna como 'Inter', 'Poppins' o 'Raleway' importada desde Google Fonts.
Iconografía	Íconos de Font Awesome o SVGs simples para botones y estados de disponibilidad.

11.4 Mockups de baja fidelidad

Un mockup de baja fidelidad (wireframe) es un boceto simple, en escala de grises y sin detalles visuales, que muestra la distribución de los elementos en cada pantalla. Su objetivo es validar la estructura y el flujo antes de escribir código. A continuación se presentan los mockups de referencia de las pantallas principales.

> 📌 **Entregable del estudiante**
> Cada grupo debe elaborar sus propios mockups de baja fidelidad (en papel, Figma, Balsamiq, Excalidraw o similar) para, como mínimo, estas pantallas: página principal, flujo de reserva, confirmación, acceso de administrador y panel de administración. Deben incluirse como imágenes en el documento final y subirse a la carpeta /docs del repositorio de GitHub.

11.4.1 Página principal (escritorio)



Figura 1. Mockup – Página principal del cliente (escritorio)

11.4.2 Flujo de reserva



Figura 2. Mockup – Modal de reserva: fecha, hora y datos del cliente

11.4.3 Versión móvil

Dado que la mayoría de los clientes accederán desde su celular, se diseña la versión móvil con un enfoque 'mobile first': lista vertical de mesas, grilla de horas de tres columnas y botones de ancho completo.



Figura 3. Mockup – Versión móvil: inicio, reserva y confirmación

11.4.4 Acceso de administrador



Figura 4. Mockup – Pantalla de acceso al panel (/admin)

11.4.5 Panel de administración: reservas

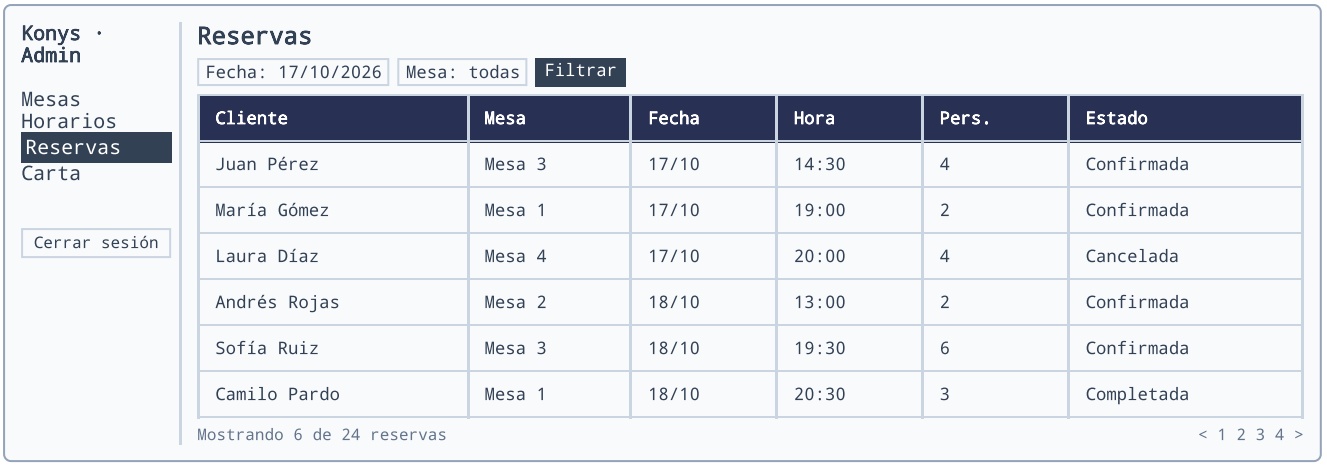


Figura 5. Mockup – Panel de administración, sección Reservas

11.4.6 Panel de administración: mesas

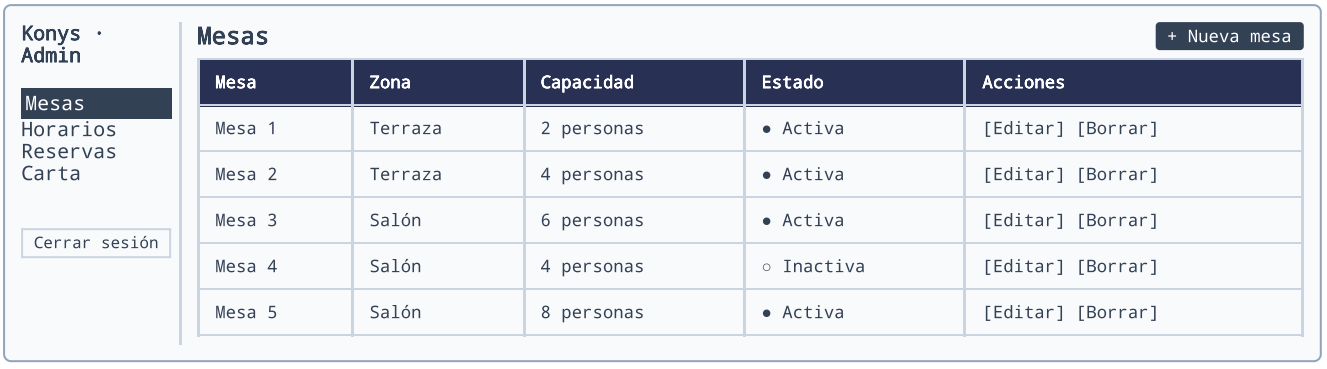


Figura 6. Mockup – Panel de administración, sección Mesas

12. Flujo del Sistema

12.1 Flujo del cliente

A continuación se describe paso a paso el flujo que sigue un cliente desde que abre la aplicación hasta que tiene su reserva confirmada:

1. El cliente accede a la URL de la aplicación en su navegador.
2. El JavaScript carga los datos de mesas desde Supabase y renderiza las cards en pantalla.
3. El cliente hace clic en la mesa que prefiere (terraza o salón).
4. El sistema muestra un selector de fecha. Solo están habilitados los días en que el restaurante atiende.
5. El cliente selecciona una fecha. El sistema consulta Supabase por las reservas existentes para ese día y esa mesa.
6. El sistema genera la grilla de slots horarios y marca como ocupados los que ya tienen reserva.
7. El cliente selecciona un slot disponible.
8. Aparece un formulario solicitando nombre, número de personas y contacto. El cliente lo completa.
9. El cliente hace clic en 'Confirmar reserva'.
10. El JavaScript verifica que el slot sigue disponible (para evitar doble reserva) y envía la solicitud a Supabase.
11. Supabase registra la nueva reserva y devuelve confirmación.
12. La aplicación muestra un mensaje de confirmación con todos los detalles de la reserva.

12.2 Flujo del administrador

1. El administrador accede a la ruta /admin.
2. Ingresa la contraseña de acceso al panel.
3. Desde el menú, selecciona la sección que desea gestionar.
4. Para registrar una mesa: completa el formulario y guarda. Supabase almacena el nuevo registro.
5. Para gestionar horarios: edita el horario por día y guarda los cambios.
6. Para ver reservas: aplica filtros y visualiza la tabla con los registros.
7. Para cambiar el estado de una mesa: activa o desactiva el toggle. El cambio se refleja en tiempo real para los clientes.

> ⚠️ **Condición de carrera (Race Condition)**
>
> Existe un riesgo técnico llamado 'condición de carrera': dos clientes reservan el mismo slot al mismo tiempo. Para mitigarlo, en el paso 10 del flujo de cliente se debe realizar una verificación en Supabase antes de confirmar la reserva. Si el slot ya fue tomado por otro cliente, se informa y se pide elegir otro. Aunque en producción se usarían transacciones atómicas, esta verificación doble es suficiente para el alcance del proyecto.

13. Plan de Desarrollo

El plan de desarrollo divide el proyecto en etapas manejables. Este tipo de planificación incremental permite entregar valor progresivamente y detectar problemas tempranamente. Cada etapa debe cerrar con al menos un commit en GitHub.

Etapa	Actividades
Etapa 1 – Análisis y diseño (~1 semana)	Leer y comprender este documento. Crear los mockups de baja fidelidad en papel o en Figma. Definir la estructura final de las colecciones en Supabase. Crear el repositorio en GitHub con la estructura de carpetas del proyecto (commit 1 y 2).
Etapa 2 – Configuración del entorno (~2 días)	Crear el proyecto en Supabase y configurar las colecciones (mesas, horarios, reservas, platos). Obtener la URL y la clave de API de Supabase. Configurar el repositorio en GitHub y conectarlo con Vercel para despliegue automático.
Etapa 3 – Desarrollo del módulo cliente (~1–2 semanas)	Crear la estructura HTML de la página principal. Aplicar estilos CSS con diseño responsivo. Implementar el JS para: cargar y mostrar las mesas, el selector de fecha/hora, la visualización de disponibilidad, el formulario de reserva y la carta digital. Conectar con Supabase para leer y escribir reservas (commit 3 y 4).
Etapa 4 – Desarrollo del módulo administrador (~1–2 semanas)	Crear la estructura HTML del panel de administración. Implementar el formulario de acceso por contraseña. Desarrollar las secciones de gestión de mesas, horarios, reservas y carta. Conectar con Supabase para todas las operaciones CRUD (commit 5).
Etapa 5 – Pruebas (~3–5 días)	Probar el flujo completo de reservas de punta a punta. Verificar responsividad en móvil y escritorio. Probar casos de error: slot ocupado, campos vacíos, personas por encima de la capacidad, sin mesas activas. Corregir errores encontrados (commit 6).
Etapa 6 – Despliegue, video y entrega (~1–2 días)	Verificar que el despliegue en Vercel funciona correctamente. Revisar que las variables de entorno de Supabase estén configuradas en Vercel. Documentar el proyecto (README en GitHub). Grabar el video explicativo y entregar la URL pública de la aplicación, el enlace del repositorio y el video.

> 💡 **Consejo de gestión del tiempo**
>
> No subestimes las etapas 1 y 2. Muchos estudiantes saltan directamente a escribir código sin planificar, lo que resulta en trabajo duplicado o en estructuras de datos incorrectas que obligan a rehacer todo desde cero. Dedica tiempo suficiente al diseño antes de codificar.

14. Consideraciones de Despliegue

14.1 Configuración de Supabase

- Crear una cuenta gratuita en supabase.com.
- Crear un nuevo proyecto y anotar la Project URL y la anon/public key.
- Crear las colecciones descritas en el modelo de datos.
- Configurar las reglas de seguridad (Row Level Security - RLS) para controlar quién puede leer y escribir datos. Para producción, se recomienda restringir escritura desde el cliente público.

14.2 Configuración de GitHub

- Crear un repositorio público (o privado con acceso al docente) con un nombre descriptivo, ej: konys-reservas-app.
- Incluir un archivo README.md con la descripción del proyecto, instrucciones de instalación y créditos.
- Usar buenas prácticas de commits: mensajes descriptivos y commits atómicos por funcionalidad.
- Incluir un archivo .gitignore para no subir archivos sensibles o de configuración local.

14.3 Configuración de Vercel

- Crear una cuenta gratuita en vercel.com y conectarla con tu cuenta de GitHub.
- Importar el repositorio de GitHub como nuevo proyecto en Vercel.
- Configurar las variables de entorno en Vercel: SUPABASE_URL y SUPABASE_ANON_KEY. Esto evita exponer credenciales en el código fuente.
- Cada push al repositorio desencadenará un nuevo despliegue automático en Vercel.

> ⚠️ **Seguridad con las credenciales**
>
> Nunca escribas directamente en el código fuente (JavaScript) la clave de API de Supabase si el repositorio es público. Usa variables de entorno de Vercel para protegerlas. En proyectos de front-end sin back-end, la clave anon de Supabase puede exponerse si no se configuran correctamente las reglas de Row Level Security.

15. Entrega en GitHub

El código fuente y la documentación del proyecto deben entregarse en un repositorio de GitHub. El historial de commits es la evidencia del proceso de desarrollo: muestra cómo evolucionó el proyecto, quién trabajó en qué y cuándo.

15.1 Requisitos de la entrega

- **Mínimo 5 commits** con mensajes descriptivos, realizados en momentos distintos del desarrollo (no se aceptan todos los commits en un solo día ni un único commit con todo el proyecto).
- Si el trabajo es en pareja o grupo, **cada integrante debe tener commits** a su nombre.
- Repositorio con nombre descriptivo (ej: konys-reservas-app), público o privado con acceso al docente.
- Archivo **README.md** con: descripción del proyecto, tecnologías, enlace de la aplicación desplegada, enlace del video e instrucciones para ejecutarlo.
- Archivo **.gitignore** y ningún secreto (claves de Supabase) dentro del código.
- Carpeta **/docs** con este documento de proyecto y las imágenes de los mockups.
- Aplicación desplegada en Vercel conectada al repositorio.
- **Evidencia:** incluye en la entrega el enlace del repositorio y una captura de la pestaña 'Commits' de GitHub (o de git log --oneline) donde se vean al menos 5 commits con sus fechas.

15.2 Estructura sugerida del repositorio

```
konys-reservas-app/
├── index.html           # Página principal (cliente)
├── admin.html           # Panel de administración
├── css/
│   └── styles.css
├── js/
│   ├── supabase.js      # Conexión con Supabase
│   ├── cliente.js       # Lógica del módulo cliente
│   └── admin.js         # Lógica del módulo administrador
├── docs/
│   ├── Documento-Proyecto.docx
│   └── mockups/         # Wireframes de baja fidelidad
├── .gitignore
└── README.md
```

15.3 Plan de commits sugerido

La siguiente tabla propone ocho commits, alineados con las etapas del plan de desarrollo. Son un mínimo de referencia: se pueden hacer más commits, pero nunca menos de cinco.

#	Mensaje del commit	Contenido
1	chore: estructura inicial del proyecto y README	Repositorio creado, carpetas, .gitignore y README con la descripción de Konys Pizza.
2	docs: agregar documento de proyecto y mockups	Documento en /docs y wireframes de baja fidelidad.
3	feat: página principal con tarjetas de mesas	HTML y CSS responsivo de la vista del cliente.
4	feat: flujo de reserva con fecha, horarios y formulario	JS del módulo cliente con validaciones y mensaje de confirmación.
5	feat: integración con Supabase	Conexión, lectura de mesas y escritura de reservas.
6	feat: panel de administración (mesas, horarios, reservas)	admin.html y admin.js con CRUD y filtros.
7	fix: validación de doble reserva y responsividad	Corrección de errores detectados en las pruebas.
8	chore: configuración de despliegue en Vercel	Variables de entorno, enlace de la app y del video en el README.

15.4 Comandos básicos

```
git init
git add .
git commit -m "chore: estructura inicial del proyecto y README"
git branch -M main
git remote add origin https://github.com/<usuario>/konys-reservas-app.git
git push -u origin main

# Para cada avance posterior:
git add .
git commit -m "feat: flujo de reserva con fecha, horarios y formulario"
git push

# Verificar el historial (mínimo 5 commits):
git log --oneline
```

16. Video Explicativo

Además del código, cada grupo debe grabar un video en el que explique el proyecto con sus propias palabras. El video permite verificar que el estudiante comprende lo que construyó y no solo que el sistema funciona.

16.1 Requisitos del video

- Duración entre **5 y 8 minutos**.
- Deben participar **todos los integrantes** del grupo, explicando su parte del trabajo (se recomienda mostrar la voz y, si es posible, el rostro).
- Grabación de pantalla con audio claro, en resolución mínima de 720p.
- **Consejo:** ensaya antes de grabar y prepara los datos de prueba en Supabase; un video corto, ordenado y con la demostración funcionando vale más que uno largo con improvisaciones.
- Publicado en YouTube (no listado) o Google Drive con acceso para el docente, y con el enlace incluido en el README y en el documento final.

16.2 Guion sugerido

Minuto	Contenido
0:00 – 0:45	Presentación del grupo y del proyecto: qué es Konys Pizza y qué problema resuelve el sistema de reservas.
0:45 – 2:00	Explicación del diseño: requisitos principales, casos de uso y mockups de baja fidelidad. Mostrar cómo los mockups se transformaron en la interfaz final.
2:00 – 4:00	Demostración del módulo cliente: ver mesas, elegir fecha y hora, reservar, ver la confirmación y consultar la carta. Probar también un caso de error (slot ocupado).
4:00 – 5:15	Demostración del módulo administrador: acceso, registro de una mesa, gestión de horarios, consulta de reservas y cambio de estado.
5:15 – 6:30	Explicación técnica: estructura del código, conexión con Supabase (mostrar las colecciones con datos reales) y despliegue en Vercel.
6:30 – 7:30	Recorrido por GitHub: repositorio, README e historial con **mínimo 5 commits**; explicar qué hizo cada integrante.
7:30 – 8:00	Conclusiones: dificultades encontradas, aprendizajes y posibles mejoras futuras.

17. Entregables del Proyecto

Al finalizar el proyecto, cada grupo debe presentar los siguientes entregables:

Entregable	Descripción
Documento de proyecto	Este documento completado, incluyendo los mockups de baja fidelidad elaborados por el grupo.
Mockups de baja fidelidad	Imágenes de las pantallas principales (cliente y administrador), también en la carpeta /docs del repositorio.
Repositorio de GitHub	Enlace al repositorio con **mínimo 5 commits**, README, .gitignore y estructura ordenada.
Aplicación desplegada	URL pública de la aplicación funcionando en Vercel y conectada a Supabase.
Video explicativo	Enlace al video de 5 a 8 minutos con la explicación y demostración del proyecto.

17.1 Mejoras futuras

Las siguientes funcionalidades están fuera del alcance del proyecto actual pero representan una evolución natural de la aplicación para versiones posteriores.

Mejora	Descripción
Autenticación de clientes	Implementar registro e inicio de sesión para que los clientes puedan ver su historial de reservas, modificarlas o cancelarlas.
Notificaciones automáticas	Enviar recordatorios de reserva por correo electrónico o WhatsApp Business API al cliente y al restaurante.
Pedidos a domicilio en línea	Integrar el servicio a domicilio de Konys Pizza con un carrito de compras y seguimiento del pedido.
Sistema de pagos	Integrar pasarelas de pago como Stripe o PayU para permitir un anticipo de la reserva en fechas de alta demanda.
Aplicación móvil	Desarrollar una PWA (Progressive Web App) o app nativa para Android e iOS que mejore la experiencia en móviles.
Panel de estadísticas	Agregar gráficas de demanda por día/hora, mesa más solicitada y tasas de no-show para apoyar decisiones gerenciales.
Sistema de calificaciones	Permitir al cliente calificar su experiencia después de la visita.
Modo oscuro	Implementar un toggle de modo oscuro/claro en la interfaz del cliente para mejorar la accesibilidad.

> 📌 **Reflexión final para el estudiante**
>
> Este proyecto cubre todas las fases de la ingeniería de software: análisis, diseño, desarrollo, pruebas y despliegue. Al completarlo, habrás aplicado conceptos clave como: levantamiento de requisitos, modelado de datos, prototipado con mockups, control de versiones con Git, consumo de APIs externas y despliegue en la nube. Cada dificultad que encuentres en el camino es una oportunidad de aprendizaje. ¡Éxito con Konys Pizza!